BCSE302P DATABASE SYSTEMS LAB · CASE STUDY 6

DBMS Project Test Suites

Railway Reservation and Passenger Service Management System — the automated test suites explained, with viva questions

---

Database: Oracle AI Database 26ai Free (release 23.26) · SQL and PL/SQL
Project files covered: parts/run_all.sql, 01_ddl.sql, 02_data.sql, 02b_seat.sql, 03_dml.sql, 04_queries.sql,
railway_case_study6.sql, tests/ (6 SQL suites, 2 Python scripts)
Outputs captured from the project database on 28 October 2026

Contents

Test Suites 3
Overview 3
T1 · 01_schema_validation.sql 4
T2 · 02_data_invariants.sql 6
T3 · 03_booking_workflow_test.sql 9
T4 · 04_cancellation_refund_test.sql 12
T5 · 05_concurrency_test.sql 15
T6 · 06_queries_verification.sql 17
T7 · run_all_tests.sql, run_tests.py and test_concurrency_race.py 19

Test Suites

The `tests/` folder of the project checks that the database works correctly. It holds six SQL test suites (T1–T6), a master runner and two Python scripts (T7).

Each test section below shows the test code, what it checks, the concepts it uses, and viva questions with answers.

Overview

The `tests/` folder holds six PL/SQL test suites, a master runner and two Python scripts. Each suite is an anonymous block with a small local **assertion procedure** (`assert_true` or `assert_zero`) that prints `[PASS]` or `[FAIL]` and counts failures. The README reports 92 passing assertions; this guide did not re-run the test suites (they modify data, then roll back or clean up).

Common pattern:

```
DECLARE
  v_failures NUMBER := 0;
  PROCEDURE assert_true(p_desc VARCHAR2, p_cond BOOLEAN) IS
  BEGIN
    IF p_cond THEN DBMS_OUTPUT.PUT_LINE('[PASS] ' || p_desc);
    ELSE DBMS_OUTPUT.PUT_LINE('[FAIL] ' || p_desc); v_failures := v_failures + 1; END IF;
  END;
BEGIN
  … checks …
END;
/
```

- A **nested (local) procedure** declared inside DECLARE exists only for that block.
- `BOOLEAN` is a PL/SQL type, so a condition such as `v_count = 1` can be passed as an argument.
- Negative tests call a procedure inside `BEGIN … EXCEPTION WHEN OTHERS THEN assert_true('…', SQLCODE = -20015); END;` so the expected error is checked by its code.

T1 · 01_schema_validation.sql

```
--  TEST SUITE 01 : SCHEMA AND OBJECT VALIDATION
-- ================================================================================
SET SERVEROUTPUT ON SIZE UNLIMITED
SET FEEDBACK OFF

DECLARE
  v_failures NUMBER := 0;
  v_count    NUMBER;

  PROCEDURE assert_true(p_desc VARCHAR2, p_cond BOOLEAN) IS
  BEGIN
    IF p_cond THEN
      DBMS_OUTPUT.PUT_LINE('[PASS] ' || p_desc);
    ELSE
      DBMS_OUTPUT.PUT_LINE('[FAIL] ' || p_desc);
      v_failures := v_failures + 1;
    END IF;
  END;
BEGIN
  DBMS_OUTPUT.PUT_LINE('--- 1. Table Existence Check ---');
  FOR t IN (
    SELECT column_value AS tbl FROM TABLE(sys.odcivarchar2list(
      'STATION','ROUTE','ROUTE_STOP','TRAIN','COACH_CLASS','COACH','SEAT',
      'TRAIN_SCHEDULE','PASSENGER','RESERVATION','PAYMENT','CANCELLATION',
      'RISK_THRESHOLD','CAPACITY_RISK_LOG'
    ))
  ) LOOP
    SELECT COUNT(*) INTO v_count FROM user_tables WHERE table_name = t.tbl;
    assert_true('Table ' || t.tbl || ' exists', v_count = 1);
  END LOOP;

  DBMS_OUTPUT.PUT_LINE('--- 2. Views Existence Check ---');
  FOR v IN (
    SELECT column_value AS vw FROM TABLE(sys.odcivarchar2list(
      'V_RESERVATION_PERFORMANCE','V_TRAIN_UTILIZATION','V_TICKET_DETAILS'
    ))
  ) LOOP
    SELECT COUNT(*) INTO v_count FROM user_views WHERE view_name = v.vw;
    assert_true('View ' || v.vw || ' exists', v_count = 1);
  END LOOP;

  DBMS_OUTPUT.PUT_LINE('--- 3. Sequences Existence Check ---');
  FOR s IN (
    SELECT column_value AS seq FROM TABLE(sys.odcivarchar2list(
      'SEQ_PNR','SEQ_PAYMENT','SEQ_CANCEL'
    ))
  ) LOOP
    SELECT COUNT(*) INTO v_count FROM user_sequences WHERE sequence_name = s.seq;
    assert_true('Sequence ' || s.seq || ' exists', v_count = 1);
  END LOOP;

  DBMS_OUTPUT.PUT_LINE('--- 4. Stored Objects Validity Check ---');
  FOR p IN (
    SELECT object_name, object_type, status FROM user_objects
    WHERE object_type IN ('PROCEDURE','FUNCTION','TRIGGER','VIEW')
    ORDER BY object_type, object_name
  ) LOOP
    assert_true(p.object_type || ' ' || p.object_name || ' is VALID', p.status = 'VALID');
  END LOOP;

  DBMS_OUTPUT.PUT_LINE('--- 5. Foreign Key Supporting Indexes Check ---');
  FOR idx IN (
    SELECT column_value AS idx_name FROM TABLE(sys.odcivarchar2list(
```

```
      'IX_RES_PASSENGER','IX_RES_FROM_STN','IX_RES_TO_STN','IX_RES_SEAT',
      'IX_RS_STATION','IX_TRAIN_ROUTE','IX_COACH_CLASS'
    ))
  ) LOOP
    SELECT COUNT(*) INTO v_count FROM user_indexes WHERE index_name = idx.idx_name;
    assert_true('Index ' || idx.idx_name || ' exists', v_count = 1);
  END LOOP;

  IF v_failures = 0 THEN
    DBMS_OUTPUT.PUT_LINE('>>> TEST SUITE 01 (SCHEMA VALIDATION): ALL CHECKS PASSED <<<');
  ELSE
    DBMS_OUTPUT.PUT_LINE('>>> TEST SUITE 01 (SCHEMA VALIDATION): ' || v_failures || ' CHECKS FAILED
<<<');
  END IF;
END;
/
```

Source: tests/01_schema_validation.sql, lines 2–79

Purpose: confirms that all 14 tables, 3 views and 3 sequences exist, that every procedure, function, trigger and view is VALID, and that the 7 FK-supporting indexes exist.

Concepts: data dictionary queries (`user_tables`, `user_views`, `user_sequences`, `user_objects`, `user_indexes`); `TABLE(sys.odcivarchar2list(…))` turns a list of literals into rows, so it can be looped over; object status VALID/INVALID (an object becomes INVALID when something it depends on changes).

What-if: renaming PASSENGER.home_city back to city would make the views INVALID, and the "is VALID" checks would fail.

HOW TO EXPLAIN THIS TO THE EXAMINER

"Suite 01 checks the schema itself: every expected table, view, sequence and index exists in the data dictionary and every stored object compiles as VALID."

Q1. When does an object become INVALID?
Short answer: When an object it depends on changes or disappears.
Detailed: Oracle recompiles it on next use if possible.
Project connection: views on renamed columns.

Q2. What does sys.odcivarchar2list do here?
Short answer: Builds a list of strings usable as a table.
Detailed: `TABLE(...)` exposes each element as a row with column COLUMN_VALUE.
Project connection: table-existence loop.

Q3. Why test index existence?
Short answer: They are part of the design (FK locking, performance).
Detailed: Losing one would not cause errors, so only a test would notice.
Project connection: Q14.

T2 · 02_data_invariants.sql

```
--  TEST SUITE 02 : DATA INTEGRITY AND BUSINESS INVARIANT CHECKS
-- ============================================================================
SET SERVEROUTPUT ON SIZE UNLIMITED
SET FEEDBACK OFF

DECLARE
  v_failures NUMBER := 0;
  v_count    NUMBER;

  PROCEDURE assert_zero(p_desc VARCHAR2, p_val NUMBER) IS
  BEGIN
    IF p_val = 0 THEN
      DBMS_OUTPUT.PUT_LINE('[PASS] ' || p_desc || ' (count = 0)');
    ELSE
      DBMS_OUTPUT.PUT_LINE('[FAIL] ' || p_desc || ' (expected 0, found ' || p_val || ')');
      v_failures := v_failures + 1;
    END IF;
  END;
BEGIN
  DBMS_OUTPUT.PUT_LINE('--- Checking Core Invariants ---');

  -- Invariant 1: No CONFIRMED/COMPLETED tickets with NULL seat
  SELECT COUNT(*) INTO v_count FROM reservation
  WHERE status IN ('CONFIRMED','COMPLETED') AND seat_id IS NULL;
  assert_zero('CONFIRMED/COMPLETED reservations without seat', v_count);

  -- Invariant 2: No WAITLIST/CANCELLED tickets with assigned seat
  SELECT COUNT(*) INTO v_count FROM reservation
  WHERE status IN ('WAITLIST','CANCELLED') AND seat_id IS NOT NULL;
  assert_zero('WAITLIST/CANCELLED reservations with non-NULL seat', v_count);

  -- Invariant 3: No duplicate seat sold on same journey
  SELECT COUNT(*) INTO v_count FROM (
    SELECT schedule_id, seat_id, COUNT(*) cnt FROM reservation
    WHERE status IN ('CONFIRMED','COMPLETED') AND seat_id IS NOT NULL
    GROUP BY schedule_id, seat_id HAVING COUNT(*) > 1
  );
  assert_zero('Duplicate active seat assignments on same journey', v_count);

  -- Invariant 4: No passenger holds multiple active tickets on same journey
  SELECT COUNT(*) INTO v_count FROM (
    SELECT schedule_id, passenger_id, COUNT(*) cnt FROM reservation
    WHERE status <> 'CANCELLED'
    GROUP BY schedule_id, passenger_id HAVING COUNT(*) > 1
  );
  assert_zero('Duplicate active tickets for same passenger on same journey', v_count);

  -- Invariant 5: Seat belongs to coach of the same train
  SELECT COUNT(*) INTO v_count FROM reservation r
  JOIN seat s ON s.seat_id = r.seat_id
  JOIN coach c ON c.coach_id = s.coach_id
  JOIN train_schedule ts ON ts.schedule_id = r.schedule_id
  WHERE c.train_no <> ts.train_no;
  assert_zero('Reservations assigned seat from wrong train', v_count);

  -- Invariant 6: Seat belongs to coach of booked class
  SELECT COUNT(*) INTO v_count FROM reservation r
  JOIN seat s ON s.seat_id = r.seat_id
  JOIN coach c ON c.coach_id = s.coach_id
  WHERE c.class_code <> r.class_code;
  assert_zero('Reservations assigned seat from wrong class', v_count);

  -- Invariant 7: Station stop sequence validity (from < to)
```

```
  SELECT COUNT(*) INTO v_count FROM reservation r
  JOIN train_schedule ts ON ts.schedule_id = r.schedule_id
  JOIN train t ON t.train_no = ts.train_no
  JOIN route_stop rs1 ON rs1.route_id = t.route_id AND rs1.station_code = r.from_station
  JOIN route_stop rs2 ON rs2.route_id = t.route_id AND rs2.station_code = r.to_station
  WHERE rs1.stop_seq >= rs2.stop_seq;
  assert_zero('Reservations with destination preceding boarding', v_count);

  -- Invariant 8: Fare computation consistency
  SELECT COUNT(*) INTO v_count FROM reservation r
  JOIN train_schedule ts ON ts.schedule_id = r.schedule_id
  JOIN train t ON t.train_no = ts.train_no
  JOIN coach_class cc ON cc.class_code = r.class_code
  JOIN route_stop rs1 ON rs1.route_id = t.route_id AND rs1.station_code = r.from_station
  JOIN route_stop rs2 ON rs2.route_id = t.route_id AND rs2.station_code = r.to_station
  WHERE r.fare <> ROUND(GREATEST(rs2.km_from_origin - rs1.km_from_origin, 50) * cc.fare_per_km + cc.rese
rvation_fee, 0);
  assert_zero('Reservations with fare mismatch against formula', v_count);

  -- Invariant 9: Every reservation has a payment record
  SELECT COUNT(*) INTO v_count FROM reservation r
  WHERE NOT EXISTS (SELECT 1 FROM payment p WHERE p.pnr = r.pnr);
  assert_zero('Reservations lacking payment record', v_count);

  -- Invariant 10: Payment amount matches reservation fare
  SELECT COUNT(*) INTO v_count FROM payment p
  JOIN reservation r ON r.pnr = p.pnr
  WHERE p.amount <> r.fare;
  assert_zero('Payments where amount <> reservation fare', v_count);

  -- Invariant 11: Every cancelled reservation has cancellation record
  SELECT COUNT(*) INTO v_count FROM reservation r
  WHERE r.status = 'CANCELLED'
  AND NOT EXISTS (SELECT 1 FROM cancellation c WHERE c.pnr = r.pnr);
  assert_zero('Cancelled reservations lacking cancellation record', v_count);

  -- Invariant 12: No non-cancelled reservation has a cancellation record
  SELECT COUNT(*) INTO v_count FROM cancellation c
  JOIN reservation r ON r.pnr = c.pnr
  WHERE r.status <> 'CANCELLED';
  assert_zero('Active reservations erroneously linked to cancellation record', v_count);

  IF v_failures = 0 THEN
    DBMS_OUTPUT.PUT_LINE('>>> TEST SUITE 02 (DATA INVARIANTS): ALL 12 CHECKS PASSED <<<');
  ELSE
    DBMS_OUTPUT.PUT_LINE('>>> TEST SUITE 02 (DATA INVARIANTS): ' || v_failures || ' CHECKS FAILED <<<');
  END IF;
END;
/
```

Source: tests/02_data_invariants.sql, lines 2–112

Purpose: 12 business **invariants** (conditions that must always be true), each written as "count the violating rows; expect 0":

1–2. Status and seat agree (also enforced by `ck_res_seat_status`). 3. No seat sold twice on a journey (GROUP BY … HAVING COUNT(*) > 1). 4. No passenger with two active tickets on a journey. 5–6. Seat belongs to the journey's train and to the booked class (trigger rule R2). 7. Boarding before destination (rule R1). 8. Every fare equals the formula (rule R4). 9. Every ticket has a payment (NOT EXISTS). 10. Payment amount = fare. 11. Every cancelled ticket has a cancellation record. 12. No active ticket has a cancellation record.

Concepts: "anti-queries" (search for violations), GROUP BY/HAVING for duplicate detection, NOT EXISTS for missing children, re-verifying trigger rules independently.

HOW TO EXPLAIN THIS TO THE EXAMINER

"Suite 02 re-checks the business rules from the data side: each check counts rows that would violate an invariant — double-booked seats, wrong-train seats, fare mismatches, tickets without payments — and passes only if the count is zero."

Q1. Why test rules already enforced by constraints?
Short answer: Defence in depth and regression testing.
Detailed: A disabled constraint, a dropped trigger, or data loaded with triggers off would be caught.
Project connection: invariants 1–8.

Q2. How do you find duplicates in SQL?
Short answer: GROUP BY the key, HAVING COUNT(*) > 1.
Detailed: Each group with more than one row is a duplicate.
Project connection: invariant 3.

Q3. How do you find parents without children?
Short answer: NOT EXISTS (or LEFT JOIN … IS NULL).
Detailed: Invariant 9 finds reservations without a payment.
Project connection: invariant 9.

T3 · 03_booking_workflow_test.sql

```
--  TEST SUITE 03 : BOOKING WORKFLOW AND BOUNDARY VALIDATION
-- ============================================================================
SET SERVEROUTPUT ON SIZE UNLIMITED
SET FEEDBACK OFF

DECLARE
  v_failures NUMBER := 0;
  v_pnr      NUMBER;
  v_status   VARCHAR2(10);
  v_count    NUMBER;

  PROCEDURE assert_true(p_desc VARCHAR2, p_cond BOOLEAN) IS
  BEGIN
    IF p_cond THEN
      DBMS_OUTPUT.PUT_LINE('[PASS] ' || p_desc);
    ELSE
      DBMS_OUTPUT.PUT_LINE('[FAIL] ' || p_desc);
      v_failures := v_failures + 1;
    END IF;
  END;
BEGIN
  DBMS_OUTPUT.PUT_LINE('--- 1. Setting up Isolated Test Schedule (ID 199) ---');
  -- Journey of 12639 (Chennai-Bengaluru) with 2 CC coaches (24 seats) and 1 EC coach (8 seats)
  DELETE FROM cancellation WHERE pnr IN (SELECT pnr FROM reservation WHERE schedule_id = 199);
  DELETE FROM payment WHERE pnr IN (SELECT pnr FROM reservation WHERE schedule_id = 199);
  DELETE FROM reservation WHERE schedule_id = 199;
  DELETE FROM train_schedule WHERE schedule_id = 199;
  INSERT INTO train_schedule VALUES (199, 12639, TRUNC(SYSDATE) + 10, 'SCHEDULED');

  DBMS_OUTPUT.PUT_LINE('--- 2. Positive Single Booking Test ---');
  book_ticket(1, 199, 'EC', 'MAS', 'SBC', 'UPI', v_pnr, v_status);
  assert_true('Booking returned valid PNR', v_pnr >= 4200001000);
  assert_true('Status is CONFIRMED', v_status = 'CONFIRMED');

  -- Verify reservation record
  SELECT COUNT(*) INTO v_count FROM reservation WHERE pnr = v_pnr AND seat_id = 301 AND status = 'CONFIR
MED';
  assert_true('Reservation created with first EC seat (301)', v_count = 1);

  -- Verify payment record
  SELECT COUNT(*) INTO v_count FROM payment p JOIN reservation r ON r.pnr = p.pnr
  WHERE p.pnr = v_pnr AND p.pay_status = 'SUCCESS' AND p.amount = r.fare;
  assert_true('Payment record created for correct fare', v_count = 1);

  DBMS_OUTPUT.PUT_LINE('--- 3. Capacity Overflow into WAITLIST Test ---');
  -- Book remaining 7 seats of EC coach (passengers 2 to 8)
  FOR p IN 2 .. 8 LOOP
    book_ticket(p, 199, 'EC', 'MAS', 'SBC', 'CARD', v_pnr, v_status);
    assert_true('Passenger ' || p || ' confirmed', v_status = 'CONFIRMED');
  END LOOP;

  -- 9th passenger (passenger 9) requests EC: should receive WAITLIST
  book_ticket(9, 199, 'EC', 'MAS', 'SBC', 'UPI', v_pnr, v_status);
  assert_true('9th passenger assigned WAITLIST status', v_status = 'WAITLIST');

  SELECT COUNT(*) INTO v_count FROM reservation WHERE pnr = v_pnr AND seat_id IS NULL AND status = 'WAIT
LIST';
  assert_true('WAITLIST reservation has NULL seat_id', v_count = 1);

  DBMS_OUTPUT.PUT_LINE('--- 4. Negative: Duplicate Active Booking on Same Journey ---');
  BEGIN
    -- Passenger 1 attempts a second booking on schedule 199
    book_ticket(1, 199, 'CC', 'MAS', 'KPD', 'UPI', v_pnr, v_status);
```

```
    assert_true('Duplicate booking should be rejected', FALSE);
  EXCEPTION
    WHEN OTHERS THEN
      assert_true('Duplicate booking rejected (ORA-20017 or ORA-00001)', SQLCODE IN (-20017, -1));
  END;

  DBMS_OUTPUT.PUT_LINE('--- 5. Negative: Nonexistent Passenger ---');
  BEGIN
    book_ticket(99999, 199, 'CC', 'MAS', 'SBC', 'UPI', v_pnr, v_status);
    assert_true('Nonexistent passenger rejected', FALSE);
  EXCEPTION
    WHEN OTHERS THEN
      assert_true('Nonexistent passenger raised ORA-20015', SQLCODE = -20015);
  END;

  DBMS_OUTPUT.PUT_LINE('--- 6. Negative: Nonexistent Schedule ---');
  BEGIN
    book_ticket(10, 9999, 'CC', 'MAS', 'SBC', 'UPI', v_pnr, v_status);
    assert_true('Nonexistent schedule rejected', FALSE);
  EXCEPTION
    WHEN OTHERS THEN
      assert_true('Nonexistent schedule raised ORA-20014', SQLCODE = -20014);
  END;

  DBMS_OUTPUT.PUT_LINE('--- 7. Negative: Closed / Completed Schedule ---');
  UPDATE train_schedule SET sched_status = 'COMPLETED' WHERE schedule_id = 199;
  BEGIN
    book_ticket(10, 199, 'CC', 'MAS', 'SBC', 'UPI', v_pnr, v_status);
    assert_true('Booking on completed schedule rejected', FALSE);
  EXCEPTION
    WHEN OTHERS THEN
      assert_true('Booking on closed schedule raised ORA-20010', SQLCODE = -20010);
  END;

  -- Cleanup test fixture
  ROLLBACK;
  DELETE FROM cancellation WHERE pnr IN (SELECT pnr FROM reservation WHERE schedule_id = 199);
  DELETE FROM payment WHERE pnr IN (SELECT pnr FROM reservation WHERE schedule_id = 199);
  DELETE FROM reservation WHERE schedule_id = 199;
  DELETE FROM train_schedule WHERE schedule_id = 199;
  COMMIT;

  IF v_failures = 0 THEN
    DBMS_OUTPUT.PUT_LINE('>>> TEST SUITE 03 (BOOKING WORKFLOW): ALL CHECKS PASSED <<<');
  ELSE
    DBMS_OUTPUT.PUT_LINE('>>> TEST SUITE 03 (BOOKING WORKFLOW): ' || v_failures || ' CHECKS FAILED <<<')
;
  END IF;
END;
/
```

Source: tests/03_booking_workflow_test.sql, lines 2–111

Purpose: creates a private journey 199 (12639, ten days ahead) and checks: the first EC booking gets seat 301 and a correct payment; seven more fill the coach; the ninth is WAITLIST with a NULL seat; a duplicate booking (-20017 or -1), an unknown passenger (-20015), an unknown journey (-20014) and a closed journey (-20010) are rejected. It then **ROLLBACK**s and deletes the fixture.

Concepts: test fixtures, positive and negative tests, checking SQLCODE, cleaning up with ROLLBACK. Because book_ticket does not commit, ROLLBACK removes all test bookings.

Q1. Why does the test use its own journey 199?
Short answer: Isolation from real data.
Detailed: Results don't depend on existing bookings and cleanup is easy.
Project connection: fixture.

Q2. Why accept -20017 or -1 for duplicates?
Short answer: Either guard may fire.
Detailed: The procedure check gives -20017; the unique index gives ORA-00001 (SQLCODE -1).
Project connection: two layers of protection.

Q3. Why can ROLLBACK undo the bookings?
Short answer: book_ticket never commits.
Detailed: The whole test runs in one transaction until the ROLLBACK.
Project connection: Q50.

T4 · 04_cancellation_refund_test.sql

```
--  TEST SUITE 04 : CANCELLATION, REFUND TIERS AND WAITLIST PROMOTION
-- ============================================================================
SET SERVEROUTPUT ON SIZE UNLIMITED
SET FEEDBACK OFF

DECLARE
  v_failures NUMBER := 0;
  v_pnr1     NUMBER;
  v_pnr2     NUMBER;
  v_pnr_wl   NUMBER;
  v_status   VARCHAR2(10);
  v_refund   NUMBER;
  v_pay_stat VARCHAR2(14);
  v_count    NUMBER;

  PROCEDURE assert_true(p_desc VARCHAR2, p_cond BOOLEAN) IS
  BEGIN
    IF p_cond THEN
      DBMS_OUTPUT.PUT_LINE('[PASS] ' || p_desc);
    ELSE
      DBMS_OUTPUT.PUT_LINE('[FAIL] ' || p_desc);
      v_failures := v_failures + 1;
    END IF;
  END;
BEGIN
  DBMS_OUTPUT.PUT_LINE('--- 1. Setting up Isolated Cancellation Test Fixture (Schedule 198) ---');
  DELETE FROM cancellation WHERE pnr IN (SELECT pnr FROM reservation WHERE schedule_id = 198);
  DELETE FROM payment WHERE pnr IN (SELECT pnr FROM reservation WHERE schedule_id = 198);
  DELETE FROM reservation WHERE schedule_id = 198;
  DELETE FROM train_schedule WHERE schedule_id = 198;
  -- Schedule departing 3 days from now (>= 48 hours bracket: 90% refund)
  INSERT INTO train_schedule VALUES (198, 12639, TRUNC(SYSDATE) + 3, 'SCHEDULED');

  -- Book all 8 EC seats so seat 301 is booked by passenger 21
  book_ticket(21, 198, 'EC', 'MAS', 'SBC', 'UPI', v_pnr1, v_status);
  FOR p IN 22 .. 28 LOOP
    book_ticket(p, 198, 'EC', 'MAS', 'SBC', 'UPI', v_pnr2, v_status);
  END LOOP;
  -- Book 9th passenger (waitlist)
  book_ticket(29, 198, 'EC', 'MAS', 'SBC', 'UPI', v_pnr_wl, v_status);
  assert_true('Passenger 29 is WAITLIST', v_status = 'WAITLIST');

  DBMS_OUTPUT.PUT_LINE('--- 2. Cancellation of CONFIRMED Ticket (>= 48 Hours) ---');
  cancel_ticket(v_pnr1, 'Test cancellation >=48h');

  -- Verify refund amount (90% of ticket fare)
  SELECT c.refund_amount, ROUND(r.fare * 0.90, 2) INTO v_refund, v_count
  FROM cancellation c JOIN reservation r ON r.pnr = c.pnr WHERE c.pnr = v_pnr1;
  assert_true('Refund amount is 90% of fare', v_refund = v_count);

  -- Verify payment status
  SELECT pay_status INTO v_pay_stat FROM payment WHERE pnr = v_pnr1;
  assert_true('Payment status updated to PARTIAL_REFUND', v_pay_stat = 'PARTIAL_REFUND');

  -- Verify reservation status
  SELECT status, seat_id INTO v_status, v_refund FROM reservation WHERE pnr = v_pnr1;
  assert_true('Reservation status updated to CANCELLED', v_status = 'CANCELLED');
  assert_true('Seat released (seat_id IS NULL)', v_refund IS NULL);

  DBMS_OUTPUT.PUT_LINE('--- 3. Automatic Waitlist Promotion Verification ---');
  -- Passenger 29 should have been promoted to seat 301
  SELECT status, seat_id INTO v_status, v_refund FROM reservation WHERE pnr = v_pnr_wl;
  assert_true('Waitlisted passenger 29 automatically promoted to CONFIRMED', v_status = 'CONFIRMED');
```

```
  assert_true('Promoted passenger received released seat 301', v_refund = 301);

  DBMS_OUTPUT.PUT_LINE('--- 4. Cancellation of WAITLIST Ticket (100% Refund) ---');
  -- Book another passenger who becomes waitlisted
  book_ticket(30, 198, 'EC', 'MAS', 'SBC', 'UPI', v_pnr_wl, v_status);
  assert_true('Passenger 30 placed on WAITLIST', v_status = 'WAITLIST');

  cancel_ticket(v_pnr_wl, 'Waitlist passenger voluntary cancellation');
  SELECT c.refund_amount, r.fare INTO v_refund, v_count
  FROM cancellation c JOIN reservation r ON r.pnr = c.pnr WHERE c.pnr = v_pnr_wl;
  assert_true('WAITLIST cancellation received 100% refund', v_refund = v_count);

  SELECT pay_status INTO v_pay_stat FROM payment WHERE pnr = v_pnr_wl;
  assert_true('WAITLIST cancellation payment status is REFUNDED', v_pay_stat = 'REFUNDED');

  DBMS_OUTPUT.PUT_LINE('--- 5. Negative: Duplicate Cancellation Rejection ---');
  BEGIN
    cancel_ticket(v_pnr1, 'Attempting second cancellation');
    assert_true('Double cancellation should be rejected', FALSE);
  EXCEPTION
    WHEN OTHERS THEN
      assert_true('Second cancellation rejected with ORA-20012', SQLCODE = -20012);
  END;

  DBMS_OUTPUT.PUT_LINE('--- 6. Negative: Cancellation on Departed Train ---');
  -- Schedule in past
  UPDATE train_schedule SET journey_date = TRUNC(SYSDATE) - 2 WHERE schedule_id = 198;
  BEGIN
    cancel_ticket(v_pnr2, 'Cancelling departed train ticket');
    assert_true('Cancellation on departed train should be rejected', FALSE);
  EXCEPTION
    WHEN OTHERS THEN
      assert_true('Departed train cancellation rejected with ORA-20013', SQLCODE = -20013);
  END;

  DBMS_OUTPUT.PUT_LINE('--- 7. Negative: Nonexistent PNR Cancellation ---');
  BEGIN
    cancel_ticket(9999999999, 'Invalid PNR');
    assert_true('Nonexistent PNR should be rejected', FALSE);
  EXCEPTION
    WHEN OTHERS THEN
      assert_true('Nonexistent PNR rejected with ORA-20016', SQLCODE = -20016);
  END;

  -- Cleanup test fixture
  ROLLBACK;
  DELETE FROM cancellation WHERE pnr IN (SELECT pnr FROM reservation WHERE schedule_id = 198);
  DELETE FROM payment WHERE pnr IN (SELECT pnr FROM reservation WHERE schedule_id = 198);
  DELETE FROM reservation WHERE schedule_id = 198;
  DELETE FROM train_schedule WHERE schedule_id = 198;
  COMMIT;

  IF v_failures = 0 THEN
    DBMS_OUTPUT.PUT_LINE('>>> TEST SUITE 04 (CANCELLATION & REFUNDS): ALL CHECKS PASSED <<<');
  ELSE
    DBMS_OUTPUT.PUT_LINE('>>> TEST SUITE 04 (CANCELLATION & REFUNDS): ' || v_failures ||
' CHECKS FAILED <<<');
  END IF;
END;
/
```

Source: tests/04_cancellation_refund_test.sql, lines 2–123

Purpose: on journey 198 (three days ahead) checks the 90% refund, PARTIAL_REFUND status, seat release, **automatic promotion** of the waitlisted passenger to seat 301, a 100% refund and REFUNDED status for a waitlist cancellation, and rejection of a second cancellation (-20012), a departed train (-20013, by moving the journey into the past) and an unknown PNR (-20016).

Concepts: end-to-end workflow tests, verifying side effects in several tables.

> **Q1. How does the test simulate a departed train?**
> **Short answer:** It moves the journey date two days into the past.
> **Detailed:** cancel_ticket then computes negative hours → -20013.
> **Project connection:** step 6.

> **Q2. Which tables does one cancellation change?**
> **Short answer:** reservation (2 rows with promotion), cancellation, payment.
> **Detailed:** All in one transaction.
> **Project connection:** Q51.

> **Q3. Why 3 days ahead?**
> **Short answer:** To land in the ≥ 48-hour tier (90%).
> **Detailed:** TRUNC(SYSDATE) + 3 plus the 07:40 departure is more than 48 hours away.
> **Project connection:** refund tiers.

T5 · 05_concurrency_test.sql

```
--  TEST SUITE 05 : CONCURRENCY, ROW LOCKING AND RACE CONDITION DEFENSE
-- ================================================================================
SET SERVEROUTPUT ON SIZE UNLIMITED
SET FEEDBACK OFF

DECLARE
  v_failures NUMBER := 0;
  v_pnr      NUMBER;
  v_status   VARCHAR2(10);
  v_count    NUMBER;

  PROCEDURE assert_true(p_desc VARCHAR2, p_cond BOOLEAN) IS
  BEGIN
    IF p_cond THEN
      DBMS_OUTPUT.PUT_LINE('[PASS] ' || p_desc);
    ELSE
      DBMS_OUTPUT.PUT_LINE('[FAIL] ' || p_desc);
      v_failures := v_failures + 1;
    END IF;
  END;
BEGIN
  DBMS_OUTPUT.PUT_LINE('--- 1. Setting up Concurrency Test Fixture (Schedule 197) ---');
  DELETE FROM cancellation WHERE pnr IN (SELECT pnr FROM reservation WHERE schedule_id = 197);
  DELETE FROM payment WHERE pnr IN (SELECT pnr FROM reservation WHERE schedule_id = 197);
  DELETE FROM reservation WHERE schedule_id = 197;
  DELETE FROM train_schedule WHERE schedule_id = 197;
  INSERT INTO train_schedule VALUES (197, 12639, TRUNC(SYSDATE) + 5, 'SCHEDULED');

  DBMS_OUTPUT.PUT_LINE('--- 2. Verifying Schedule-Level Lock Acquisition (FOR UPDATE) ---');
  -- When book_ticket executes, it locks train_schedule row for that journey
  DECLARE
    v_locked_status VARCHAR2(10);
  BEGIN
    SELECT sched_status INTO v_locked_status
    FROM train_schedule
    WHERE schedule_id = 197
    FOR UPDATE NOWAIT;
    assert_true('Acquired row-level lock on train_schedule without blocking', v_locked_status = 'SCHEDUL
ED');
  END;

  DBMS_OUTPUT.PUT_LINE('--- 3. Verifying Final Safeguard: Unique Constraint UX_RES_SEAT_JOURNEY ---');
  -- Book seat 301 on schedule 197
  INSERT INTO reservation (pnr, passenger_id, schedule_id, class_code, seat_id, from_station,
to_station, booking_date, status)
  VALUES (4299999001, 40, 197, 'EC', 301, 'MAS', 'SBC', TRUNC(SYSDATE), 'CONFIRMED');

  -- Attempt simultaneous direct insertion of same seat (simulating concurrent race)
  BEGIN
    INSERT INTO reservation (pnr, passenger_id, schedule_id, class_code, seat_id, from_station, to_stati
on, booking_date, status)
    VALUES (4299999002, 41, 197, 'EC', 301, 'MAS', 'SBC', TRUNC(SYSDATE), 'CONFIRMED');
    assert_true('Simultaneous double booking of same seat must be blocked', FALSE);
  EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
      assert_true('Database-level UX_RES_SEAT_JOURNEY strictly prevented duplicate seat assignment', TRU
E);
  END;

  DBMS_OUTPUT.PUT_LINE('--- 4. Verifying Passenger Uniqueness Safeguard: UX_RES_PASS_JOURNEY ---');
  -- Passenger 40 attempts another active reservation on schedule 197
  BEGIN
    INSERT INTO reservation (pnr, passenger_id, schedule_id, class_code, seat_id, from_station, to_stati
```

```
on, booking_date, status)
    VALUES (4299999003, 40, 197, 'EC', 302, 'MAS', 'SBC', TRUNC(SYSDATE), 'CONFIRMED');
    assert_true('Same passenger second active booking must be blocked', FALSE);
  EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
      assert_true('Database-level UX_RES_PASS_JOURNEY strictly prevented duplicate passenger ticket', TR
UE);
  END;

  -- Cleanup test fixture
  ROLLBACK;
  DELETE FROM cancellation WHERE pnr IN (SELECT pnr FROM reservation WHERE schedule_id = 197);
  DELETE FROM payment WHERE pnr IN (SELECT pnr FROM reservation WHERE schedule_id = 197);
  DELETE FROM reservation WHERE schedule_id = 197;
  DELETE FROM train_schedule WHERE schedule_id = 197;
  COMMIT;

  IF v_failures = 0 THEN
    DBMS_OUTPUT.PUT_LINE('>>> TEST SUITE 05 (CONCURRENCY & LOCKING): ALL CHECKS PASSED <<<');
  ELSE
    DBMS_OUTPUT.PUT_LINE('>>> TEST SUITE 05 (CONCURRENCY & LOCKING): ' || v_failures || ' CHECKS FAILED
<<<');
  END IF;
END;
/
```

Source: tests/05_concurrency_test.sql, lines 2–82

Purpose: proves that the journey row can be locked (`FOR UPDATE NOWAIT` fails immediately instead of waiting if someone else holds the lock), and that the two unique indexes reject a second ticket for the same seat and a second active ticket for the same passenger (`WHEN DUP_VAL_ON_INDEX`).

Concepts: row locks, NOWAIT (ORA-00054 if locked), named exceptions, database-level guarantees under concurrency.

> **Q1. FOR UPDATE vs FOR UPDATE NOWAIT?**
> **Short answer:** Wait vs fail immediately.
> **Detailed:** NOWAIT raises ORA-00054 if another session holds the lock.
> **Project connection:** step 2.

> **Q2. Can a single session test true concurrency?**
> **Short answer:** Not fully.
> **Detailed:** This suite simulates it with direct inserts; test_concurrency_race.py uses two real sessions.
> **Project connection:** Python test.

> **Q3. What is DUP_VAL_ON_INDEX?**
> **Short answer:** The PL/SQL name for ORA-00001.
> **Detailed:** Raised when a unique index or constraint is violated.
> **Project connection:** steps 3–4.

T6 · 06_queries_verification.sql

```
--  TEST SUITE 06 : ANALYTICAL VIEWS, QUERIES AND DECISION SUPPORT VERIFICATION
-- ================================================================================
SET SERVEROUTPUT ON SIZE UNLIMITED
SET FEEDBACK OFF

DECLARE
  v_failures NUMBER := 0;
  v_count    NUMBER;
  v_val      NUMBER;
  v_txt      VARCHAR2(50);

  PROCEDURE assert_true(p_desc VARCHAR2, p_cond BOOLEAN) IS
  BEGIN
    IF p_cond THEN
      DBMS_OUTPUT.PUT_LINE('[PASS] ' || p_desc);
    ELSE
      DBMS_OUTPUT.PUT_LINE('[FAIL] ' || p_desc);
      v_failures := v_failures + 1;
    END IF;
  END;
BEGIN
  DBMS_OUTPUT.PUT_LINE('--- 1. View v_reservation_performance Verification ---');
  SELECT COUNT(*) INTO v_count FROM v_reservation_performance;
  assert_true('v_reservation_performance returns rows for all active schedules (>= 27)', v_count >= 27);

  -- Check occupancy calculation range [0, 100]
  SELECT COUNT(*) INTO v_count FROM v_reservation_performance
  WHERE occupancy_pct < 0 OR occupancy_pct > 100;
  assert_true('All journey occupancy rates within valid 0-100% bounds', v_count = 0);

  DBMS_OUTPUT.PUT_LINE('--- 2. View v_train_utilization Verification ---');
  SELECT COUNT(*) INTO v_count FROM v_train_utilization;
  assert_true('v_train_utilization returns exactly 9 trains', v_count = 9);

  -- Check top utilized trains (12639 and 12027 should have occupancy >= 96%)
  SELECT occupancy_pct INTO v_val FROM v_train_utilization WHERE train_no = 12639;
  assert_true('Train 12639 occupancy is 96.88%', v_val = 96.88);

  SELECT occupancy_pct INTO v_val FROM v_train_utilization WHERE train_no = 12677;
  assert_true('Train 12677 occupancy is 39.84%', v_val = 39.84);

  DBMS_OUTPUT.PUT_LINE('--- 3. View v_ticket_details Verification ---');
  SELECT COUNT(*) INTO v_count FROM v_ticket_details;
  assert_true('v_ticket_details returns all reservation tickets (>= 768)', v_count >= 768);

  DBMS_OUTPUT.PUT_LINE('--- 4. Decision Support Function fn_capacity_risk Verification ---');
  -- Train 12639: 96.88% occ, 1.96% cancel, 104.17% demand -> CAPACITY REQUIRED
  v_txt := fn_capacity_risk(96.88, 1.96, 104.17);
  assert_true('Rule for 12639 evaluates to CAPACITY REQUIRED', v_txt = 'CAPACITY REQUIRED');

  -- Train 12027: 96.88% occ, 12.50% cancel, 102.08% demand -> HIGH DEMAND (high cancel rate frees
seats)
  v_txt := fn_capacity_risk(96.88, 12.50, 102.08);
  assert_true('Rule for 12027 evaluates to HIGH DEMAND', v_txt = 'HIGH DEMAND');

  -- Train 12677: 39.84% occ, 5.56% cancel, 39.84% demand -> NORMAL
  v_txt := fn_capacity_risk(39.84, 5.56, 39.84);
  assert_true('Rule for 12677 evaluates to NORMAL', v_txt = 'NORMAL');

  DBMS_OUTPUT.PUT_LINE('--- 5. Capacity Risk Procedure Execution Verification ---');
  prc_capacity_risk_report;
  SELECT COUNT(*) INTO v_count FROM capacity_risk_log WHERE run_date = TRUNC(SYSDATE);
  assert_true('capacity_risk_log has 9 entries logged for today', v_count = 9);
```

```
  SELECT COUNT(*) INTO v_count FROM capacity_risk_log
  WHERE run_date = TRUNC(SYSDATE) AND risk_status = 'CAPACITY REQUIRED';
  assert_true('Exactly 2 trains flagged as CAPACITY REQUIRED (12639 & 12637)', v_count = 2);

  IF v_failures = 0 THEN
    DBMS_OUTPUT.PUT_LINE('>>> TEST SUITE 06 (ANALYTICAL VIEWS & RISK): ALL CHECKS PASSED <<<');
  ELSE
    DBMS_OUTPUT.PUT_LINE('>>> TEST SUITE 06 (ANALYTICAL VIEWS & RISK): ' || v_failures || ' CHECKS
FAILED <<<');
  END IF;
END;
/
```

Source: tests/06_queries_verification.sql, lines 2–75

Purpose: checks the views (row counts, occupancy within 0–100%, exact values 96.88% for 12639 and 39.84% for 12677), the function on three known cases, and that the procedure logs 9 trains with exactly 2 CAPACITY REQUIRED.

Concepts: verifying analytical results against expected values (regression tests for reports).

INTERACTION WITH THE DEMO

The check "12677 occupancy is 39.84%" includes demo journey 130 (51 occupied / 128 capacity). If the demo were changed, this expected value would also change.

Q1. Why test exact numbers?
Short answer: To detect any change in report logic.
Detailed: A refactor that changes results by mistake would fail the test (as when this guide's simplifications were checked).
Project connection: 96.88, 39.84.

Q2. Does calling prc_capacity_risk_report in a test have side effects?
Short answer: Yes, it writes and commits the log.
Detailed: The procedure's COMMIT makes the test's log rows permanent.
Project connection: Q86.

Q3. Why check 0–100% occupancy?
Short answer: A sanity bound.
Detailed: Over 100% would mean a seat sold twice or a capacity error.
Project connection: v_reservation_performance.

T7 · run_all_tests.sql, run_tests.py and test_concurrency_race.py

- **run_all_tests.sql** runs the six suites with `@@`, like run_all.sql.
- **run_tests.py** starts `sqlplus -S` with `subprocess`, feeds it `@run_all_tests.sql`, then uses regular expressions to count `[PASS]` and `[FAIL]` lines and exits with code 1 if anything failed (useful in automated pipelines).
- **test_concurrency_race.py** prepares journey 196 with exactly one free EC seat (308), then starts **two Python threads**, each opening its own SQL*Plus session and calling `book_ticket` for a different passenger at the same time. It then checks that exactly one passenger is CONFIRMED on seat 308, the other is WAITLIST with no seat, and no seat is duplicated. This is a real race-condition test across two database sessions.

CREDENTIALS

Both Python scripts read the connection details from environment variables (`DB_USER`, `DB_PASS`, …) but fall back to **hard-coded default test credentials** if the variables are not set. In a real project, credentials should never be stored in source code. (The values are deliberately not reproduced in this guide.)

Q1. How does the race test create concurrency?
Short answer: Two threads, two separate database sessions.
Detailed: Each thread runs its own sqlplus process; both call book_ticket for the last seat simultaneously.
Project connection: test_concurrency_race.py.

Q2. What result proves correctness?
Short answer: One CONFIRMED on seat 308, one WAITLIST, no duplicate seat.
Detailed: The schedule lock serialises the two calls; the unique index is the backup.
Project connection: CONCURRENCY_VERIFIED.

Q3. Is there SQL injection risk in the Python scripts?
Short answer: Not in practice.
Detailed: They build SQL with f-strings, but only from integers defined in the script, never from user input; with user input, bind variables would be required.
Project connection: worker_book().